Cornell Bowers CIS
Computer Science

Applied High-Performance and Parallel Computing

Single Processor Architecture and Introduction to GEMM (MatMul)

Professor Giulia Guidi

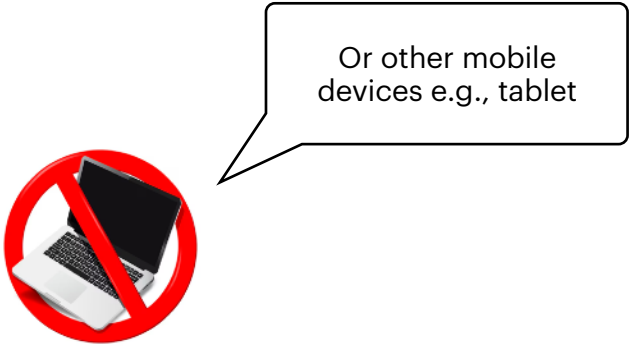
01/29/2026

A number of slides were taken from or inspired by slides by David Bindel, Jim Demmel, and Yoav Etsion

Plan for Today

- Single Core Architecture + Intro to MatMul (~35 minutes)
- HW1 Recitation by Julian Bellavita (TA) (~40 minutes)

no smartphones



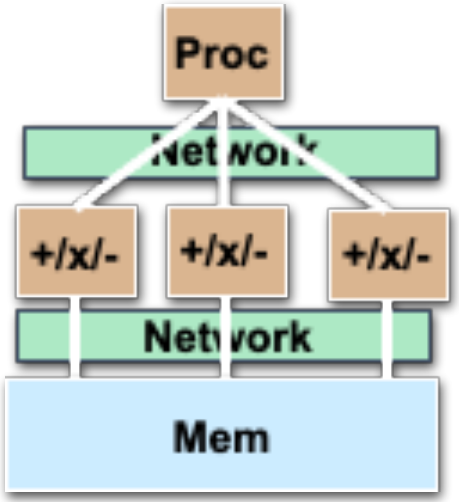
no laptop

Why? Research shows that they slow us down in a learning environment.
I recommend **paper and pen** for taking notes

Credit to: https://users.cs.utah.edu/~pandey/courses/cs6530/fall23/index.html

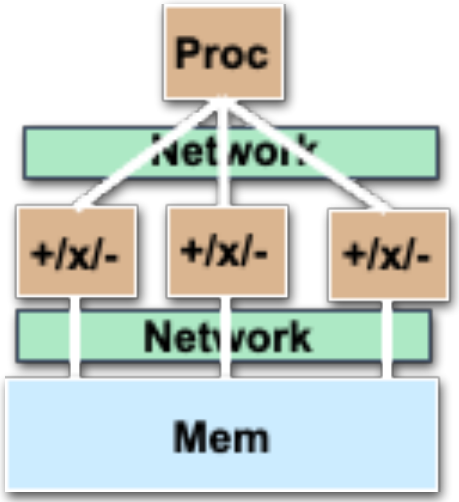
Plan for Today

- Single Core Architecture + Intro to MatMul (~35 minutes)
- HW1 Recitation by Julian Bellavita (TA) (~40 minutes)



One core is not so serial!

- Single Core Architecture + Intro to MatMul (~35 minutes)
- HW1 Recitation by Julian Bellavita (TA) (~40 minutes)



Parallel Processing at the Laundromat

It's Sunday night, it's snowing, and you need to do laundry

Once you recovered from the Sunday night existential crisis, you decide to start:

- **3** stages to laundry: **wash**, **dry**, and **fold**
- **3** loads: darks, lights, and underwear

Parallel Processing at the Laundromat

Serial Laundry

1	2	3	4	5	6	7	8	9
wash	dry	fold						
			wash	dry	fold			
						wash	dry	fold

Pipelined Laundry

1	2	3	4	5	
wash	dry	fold			Dinner?
	wash	dry	fold		Dog walk?
		wash	dry	fold	CS5220 slides?

Pipelining

Pipelining **improves *bandwidth***, but **not *latency:***

I can wash and dry simultaneously, but the action itself takes the same amount of time, i.e., pipelining does not reduce time delay of a single action

Potential speedup = number of stages

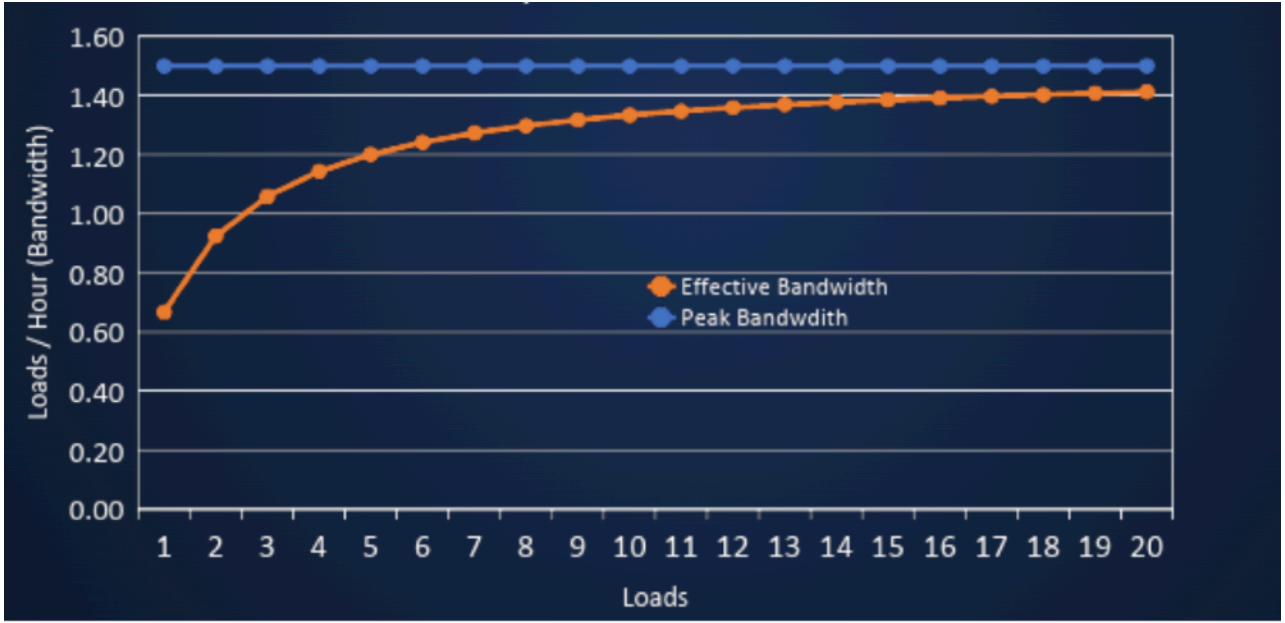
This only happens if the pipeline stages are all of equal length and have enough loads to saturate the bandwidth

Bandwidth is **limited by the slowest pipeline stage**

If loads have different lengths, the slowest determines the bandwidth (others wait)

Laundry Bandwidth

- The system bandwidth is 1 load per 40 minutes (1.5 loads/hour)
- The effective bandwidth depends on available loads that can be done in parallel



Plot by Jim Demmel

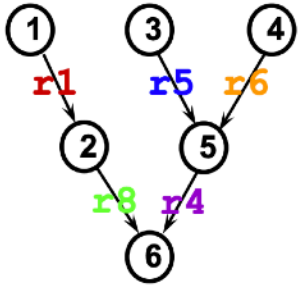
Out-of-Order in a Nutshell

- The CPU examines a sliding windows of consecutive instructions:
  - The instruction “window”
- Ready instructions get picked up from the window
- They are executed out of order based on the **“data flow graph”**
  - Limit: data dependencies
- The results are committed to the machine state (memory and register file) in original **program order**
- User is unaware (except that the program runs faster)

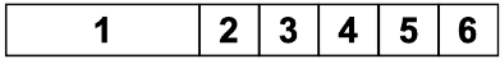
Out-of-Order in a Nutshell

```
(1) r1 ← r4 / r7      /* assume division takes 20 cycles */
(2) r8 ← r1 + r2
(3) r5 ← r5 + 1
(4) r6 ← r6 - r3
(5) r4 ← r5 + r6
(6) r7 ← r8 * r4
```

Data Flow Graph

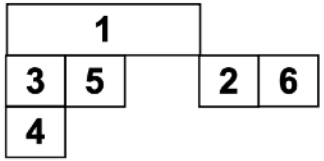


In-order execution



In-order (2-way superscalar)

Out-of-order execution



Take Away

Tuesday next week we will dive into **SIMD parallelism**

- **Memory operations are not all the same!**
  - Registers and caches lead to variable access times
- **Instructions are non-obvious!**
  - Pipelining allows instructions to overlap
  - Functional units run in parallel (and out of order)
  - Instructions take different amounts of time

Our goal: enough understanding to help the compiler out

Example

Matrix Multiply

```
do i = 1, n
  do j = 1, n
    c(i, j) = 0
    do k = 1, n
      c(i, j) = c(i, j) + a(i, k) * b(k, j)
```

Why MatMul?

- An important kernel in many problems:
  - Dense linear algebra is popular a motif in many computational areas
  - Closely related to other algorithms, e.g., transitive closure on a graph
  - Dominates training time in deep learning (e.g., CNNs)

Example

Matrix Multiply

```
do i = 1, n
  do j = 1, n
    c(i, j) = 0
    do k = 1, n
      c(i, j) = c(i, j) + a(i, k) * b(k, j)
```

Why MatMul?

- Optimization ideas can be used in other problems
- The best case for optimization payoff
- The most-studied algorithm in high performance computing

Example

Matrix Multiply

```
do i = 1, n
  do j = 1, n
    c(i, j) = 0
    do k = 1, n
      c(i, j) = c(i, j) + a(i, k) * b(k, j)
```

Performance Estimate: how fast should this run?

- Counting FLOPs: $2n^3$
  - 3 nested loops, each with n iterations
  - 1 multiply and 1 add in each inner iteration
- For n = 100, 2×10^6 operations, or about 1 millisecond on a 2GHz processor
- For n = 1000, 2×10^9 operations, or about 1 second

Example

Matrix Multiply

```
do i = 1, n
  do j = 1, n
    c(i, j) = 0
    do k = 1, n
      c(i, j) = c(i, j) + a(i, k) * b(k, j)
```

Performance Estimate: how fast should this run?

- For n = 100, 1818 MFLOP operations in 1.1 millisecond
- For n = 1000, 335 MFLOP operations in 6 seconds

- For n = 100, 2×10^6 operations, or about 1 millisecond on a 2GHz processor
- For n = 1000, 2×10^9 operations, or about 1 second

Example

Matrix Multiply

```
do i = 1, n
  do j = 1, n
    c(i, j) = 0
    do k = 1, n
      c(i, j) = c(i, j) + a(i, k) * b(k, j)
```

Performance Estimate: how fast should this run?

- For n = 100, 1818 MFLOP operations in 1.1 millisecond
- For n = 1000, 335 MFLOP operations in 6 seconds

Obvious expression of algorithms are **not** transformed into leading performance!

A Simple Performance Model

Matrix Multiply

Assumption 1: two level of memory hierarchy, fast and slow

Assumption 2: data initially in slow memory

- m = number of memory elements (words) moved between fast and slow memory
- t_m = time per slow memory operation (inverse bandwidth in the best case)
- f = number of arithmetics operations
- t_f = time per arithmetic operation $\ll t_m$
- $CI = f / m$ = average number of FLOPs per slow memory access

Computational Intensity (CI) = key to algorithm efficiency!

The **minimum amount of time possible** = $f * t_f$ if data in fast memory

The **actual time** is:

$$f * t_f + m * t_m = f * t_f * (1 + t_m / t_f * 1 / CI)$$

Machine Balance = key to machine efficiency!

The **larger CI** is, the **closer to the minimum** $f * t_f$ we can get

Machine Balance and Computational Intensity

Machine Balance:

Characteristic of the machine

$$\text{Balance} = \frac{\text{Peak DP FLOP/s}}{\text{Peak Bandwidth}}$$

Computation or Arithmetic Intensity (CI or AI):

Characteristic of both the code and the machine (cache effect)

$$\text{CI} = \frac{\text{FLOPs Performed}}{\text{Data Moved}}$$

Machine Balance and Computational Intensity

Machine Balance:

Characteristic of the machine

$$\text{Balance} = \frac{\text{Peak DP FLOP/s}}{\text{Peak Bandwidth}}$$

Computation or Arithmetic Intensity (CI or AI):

Characteristic of both the code and the machine (cache effect)

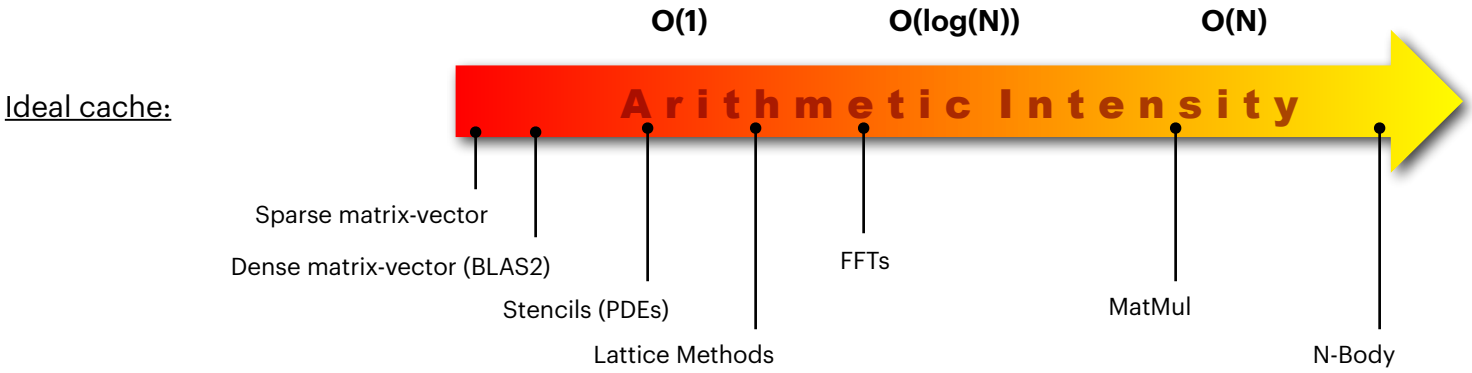
$$\text{CI} = \frac{\text{FLOPs Performed}}{\text{Data Moved}}$$

Operation	FLOPs	Data	CI
Dot Prod	$O(n)$	$O(n)$	$O(1)$
Mat Vec	$O(n^2)$	$O(n^2)$	$O(1)$
MatMul	$O(n^3)$	$O(n^2)$	$O(n)$
N-Body	$O(n^2)$	$O(n)$	$O(n)$
FFT	$O(n \log n)$	$O(n)$	$O(\log n)$

If ideal (infinite) cache!

Computational Intensity

- Can look at computational intensity as a spectrum
- Constants (at least leading constants) will matter



A Simple Performance Model

Naïve Matrix Multiply

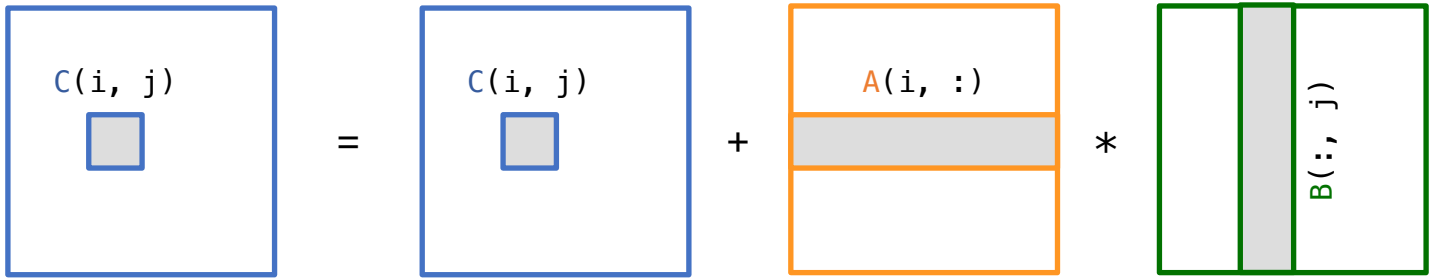
```
do i = 1, n
   {read row i of A into fast memory}
   do j = 1, n
      {read C(i, j) into fast memory}
      {read column j of B into fast memory }
      do k = 1, n
         C(i, j) = C(i, j) + A(i, k) * B(k, j)
         {write back C(i, j) into slow memory}
```

$f = 2n^3$ arithmetics ops, how many slow memory?

n^2 to read each row of A once

$2n^2$ to read and write each element of C once

n^3 to read each column of B n times



A Simple Performance Model

Naïve Matrix Multiply

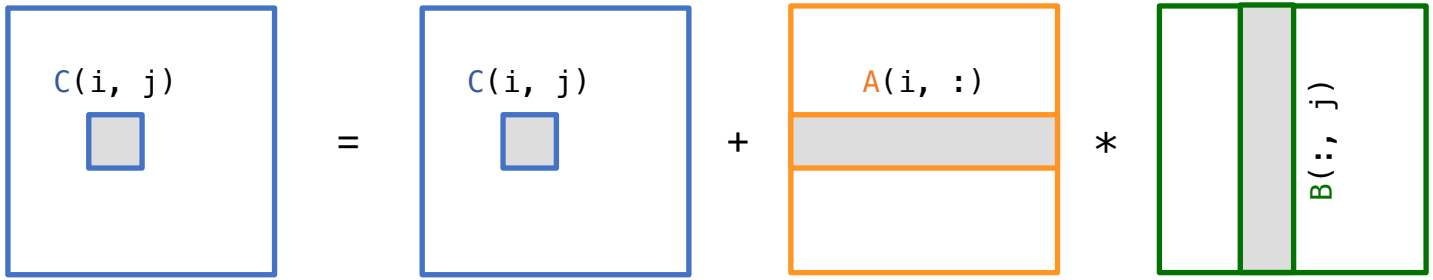
```
do i = 1, n
   {read row i of A into fast memory}
   do j = 1, n
      {read C(i, j) into fast memory}
      {read column j of B into fast memory }
      do k = 1, n
         C(i, j) = C(i, j) + A(i, k) * B(k, j)
         {write back C(i, j) into slow memory}
```

$f = 2n^3$ arithmetics, $m = 3n^2 + n^3$ slow memory

n^2 to read each row of A once

$2n^2$ to read and write each element of C once

n^3 to read each column of B n times



A Simple Performance Model

Naïve Matrix Multiply

```
do i = 1, n
   {read row i of A into fast memory}
   do j = 1, n
      {read C(i, j) into fast memory}
      {read column j of B into fast memory }
      do k = 1, n
         C(i, j) = C(i, j) + A(i, k) * B(k, j)
      {write back C(i, j) into slow memory}
```

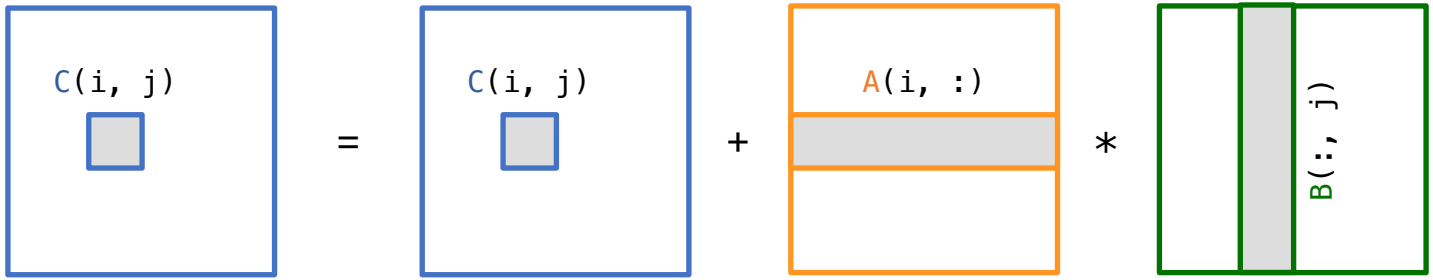
$f = 2n^3$ arithmetic, $m = 3n^2 + n^3$ slow memory

Computational Intensity (CI) = f / m

$= 2n^3 / (3n^2 + n^3)$

≈ 2

Low CI: matmul **limited** by slow memory speed



A Simple Performance Model

Blocked Matrix Multiply

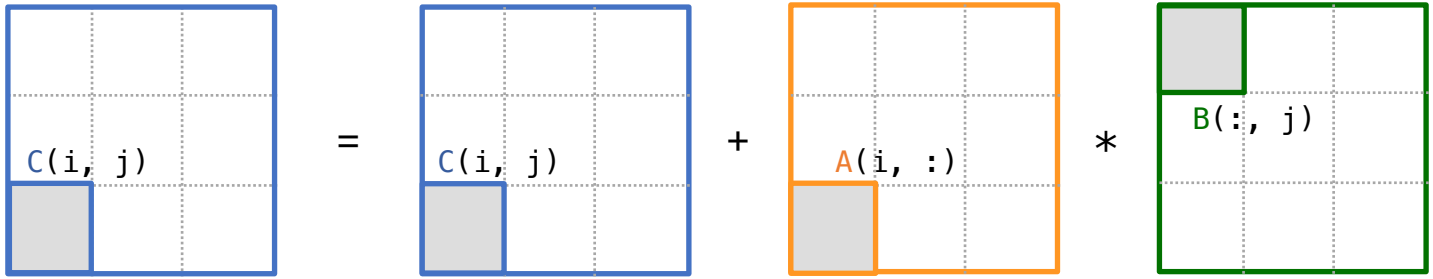
```
do i = 1, N
  do j = 1, N
    {read C(i, j) into fast memory}
    do k = 1, N
    {read block A(i, k) into fast memory}
    {read block B(k, j) into fast memory}
      C(i, j) = C(i, j) + A(i, k) * B(k, j) {matmul on a block}
      {write back C(i, j) into slow memory}
```

b = block size = n/N where N is block number

$2N^2 * b^2 = 2n^2$ to read and write each element of C

$N^3 * b^2 = N*n^2$ to read each block of A N^3 times

$N*n^2$ to read each block of B N^3 times



A Simple Performance Model

Blocked Matrix Multiply

```
do i = 1, N
  do j = 1, N
    {read C(i, j) into fast memory}
    do k = 1, N
    {read block A(i, k) into fast memory}
    {read block B(k, j) into fast memory}
      C(i, j) = C(i, j) + A(i, k) * B(k, j) {matmul on a block}
    {write back C(i, j) into slow memory}
```

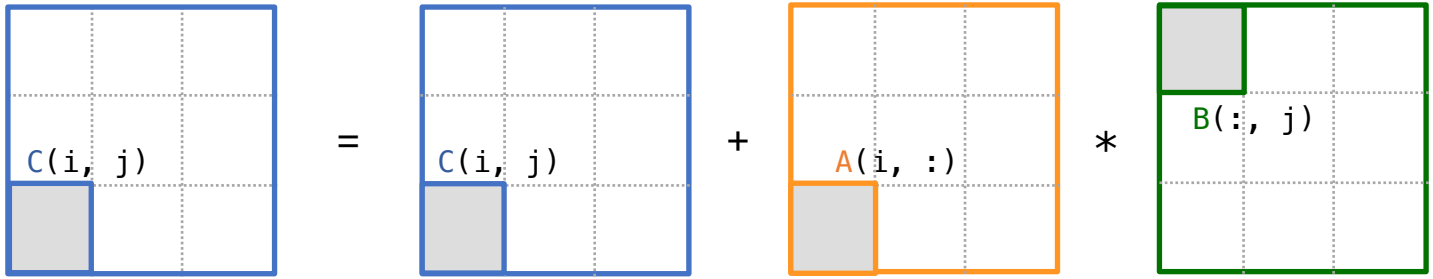
b = block size = n/N where N is block number

Computational Intensity (CI) = f / m

$= 2n^3 / ((2N + 2) * n^2)$

$\approx n / N = b$ for large n

Better!



A Simple Performance Model

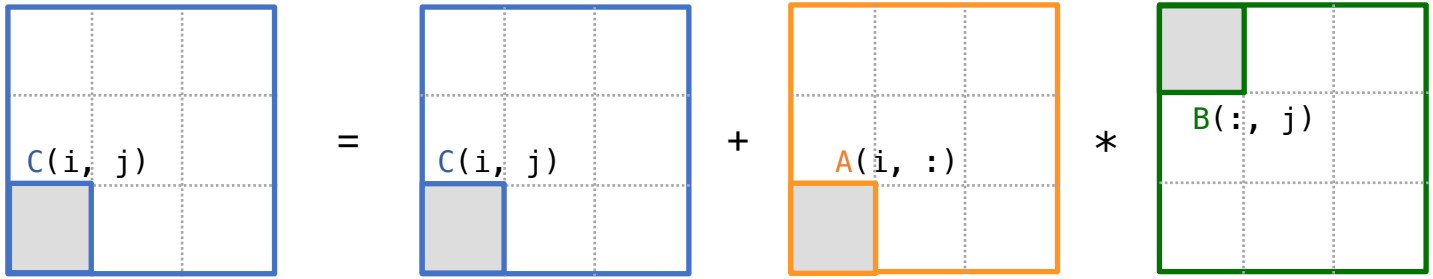
Blocked (Tiled) Matrix Multiply

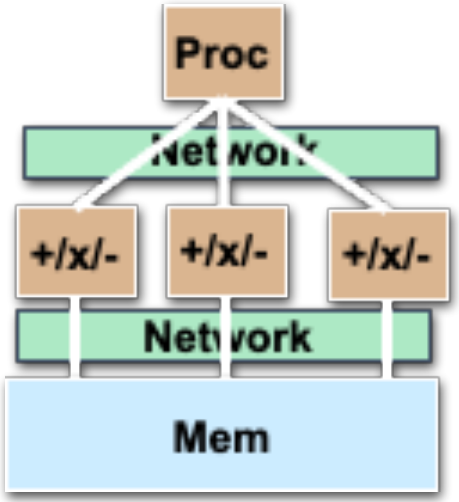
```
do i = 1, N
  do j = 1, N
    {read C(i, j) into fast memory}
    do k = 1, N
    {read block A(i, k) into fast memory}
    {read block B(k, j) into fast memory}
      C(i, j) = C(i, j) + A(i, k) * B(k, j) {matmul on a block}
    {write back C(i, j) into slow memory}
```

Computational Intensity (CI) = b for large n

How large can we make b? Let's assume our fast memory has size M:

$b \le \sqrt{M/3}$ since M must hold 3 b-by-b blocks



Plan for Today

- Single Core Architecture + Intro to MatMul (~35 minutes)
- HW1 Recitation by Julian Bellavita (TA) (~40 minutes)